```
#Date : 31/07/2026
#DT Batch 3
#Sahil Tagade
#44
#12410016
```

```
# Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the
#effect of different learning rates and the number of epochs on model performance.
```

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

#wine dataset
from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score
)

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

from tensorflow.keras.optimizers import Adam

import warnings
warnings.filterwarnings("ignore")

print("Libraries Imported Successfully!")
```

```
Libraries Imported Successfully!
```

```
wine = load_wine()

X = wine.data

y = wine.target

print("Wine Dataset Loaded Successfully!")
```

```
Wine Dataset Loaded Successfully!
```

```
df = pd.DataFrame(
    X,
    columns=wine.feature_names
)

df["Wine Class"] = y

print("First 5 Rows of Dataset")

df.head()
```

First 5 Rows of Dataset

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	col
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	

```
print("Shape of Dataset")
print(df.shape)

print("\n")

print("Column Names")

print(df.columns)

print("\n")

print("Dataset Information")

df.info()
```

```
Shape of Dataset
(178, 14)


Column Names
Index(['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium',
       'total_phenols', 'flavanoids', 'nonflavanoid_phenols',
       'proanthocyanins', 'color_intensity', 'hue',
       'od280/od315_of_diluted_wines', 'proline', 'Wine Class'],
      dtype='object')


Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 14 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   alcohol                       178 non-null    float64
 1   malic_acid                    178 non-null    float64
 2   ash                           178 non-null    float64
 3   alcalinity_of_ash             178 non-null    float64
 4   magnesium                     178 non-null    float64
 5   total_phenols                 178 non-null    float64
 6   flavanoids                    178 non-null    float64
 7   nonflavanoid_phenols          178 non-null    float64
 8   proanthocyanins               178 non-null    float64
 9   color_intensity               178 non-null    float64
 10  hue                           178 non-null    float64
 11  od280/od315_of_diluted_wines  178 non-null    float64
 12  proline                       178 non-null    float64
 13  Wine Class                    178 non-null    int64
dtypes: float64(13), int64(1)
memory usage: 19.6 KB
```

```
df.describe()
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanth
count	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	17
mean	13.000618	2.336348	2.366517	19.494944	99.741573	2.295112	2.029270	0.361854	
std	0.811827	1.117146	0.274344	3.339564	14.282484	0.625851	0.998859	0.124453	
min	11.030000	0.740000	1.360000	10.600000	70.000000	0.980000	0.340000	0.130000	
25%	12.362500	1.602500	2.210000	17.200000	88.000000	1.742500	1.205000	0.270000	
50%	13.050000	1.865000	2.360000	19.500000	98.000000	2.355000	2.135000	0.340000	
75%	13.677500	3.082500	2.557500	21.500000	107.000000	2.800000	2.875000	0.437500	
max	14.830000	5.800000	3.230000	30.000000	162.000000	3.880000	5.080000	0.660000	

```
print("Missing Values")

print(df.isnull().sum())

print("\n")

print("Duplicate Rows")

print(df.duplicated().sum())
```

```
Missing Values
alcohol                         0
malic_acid                      0
ash                             0
alcalinity_of_ash               0
magnesium                       0
total_phenols                   0
flavanoids                      0
nonflavanoid_phenols            0
proanthocyanins                 0
color_intensity                 0
hue                             0
od280/od315_of_diluted_wines    0
proline                         0
Wine Class                      0
dtype: int64


Duplicate Rows
0
```

```
plt.figure(figsize=(7,5))

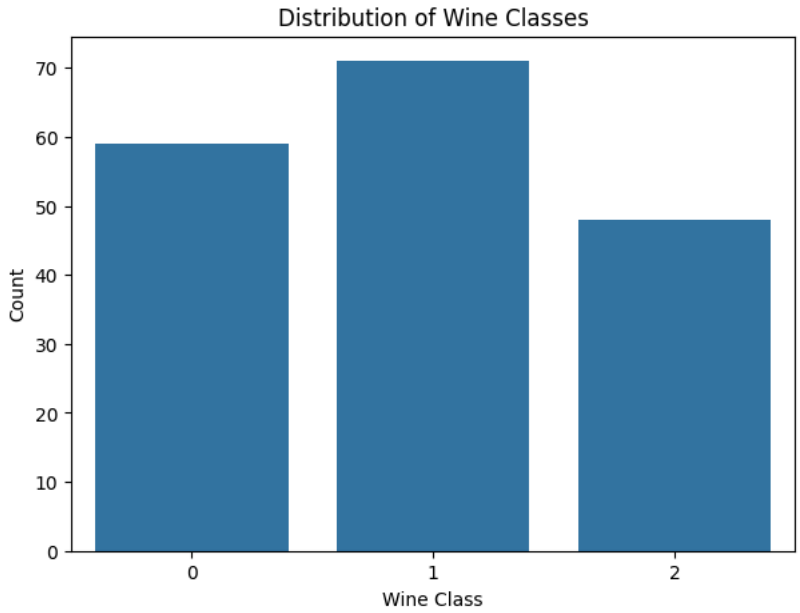
sns.countplot(
    x="Wine Class",
    data=df
)

plt.title("Distribution of Wine Classes")

plt.xlabel("Wine Class")

plt.ylabel("Count")

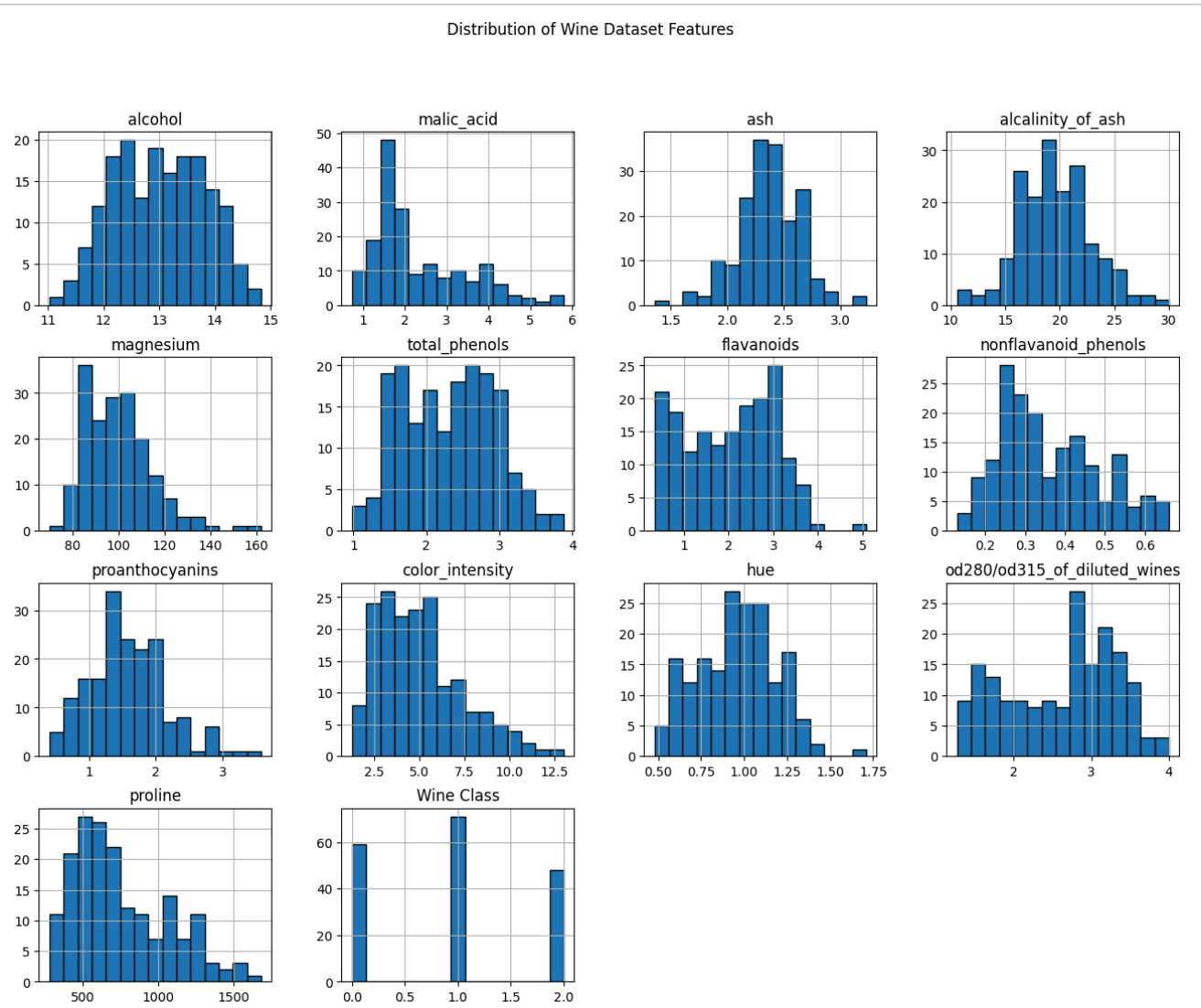
plt.show()
```




```
df.hist(
    figsize=(16,12),
    bins=15,
    edgecolor='black'
)

plt.suptitle("Distribution of Wine Dataset Features")

plt.show()
```



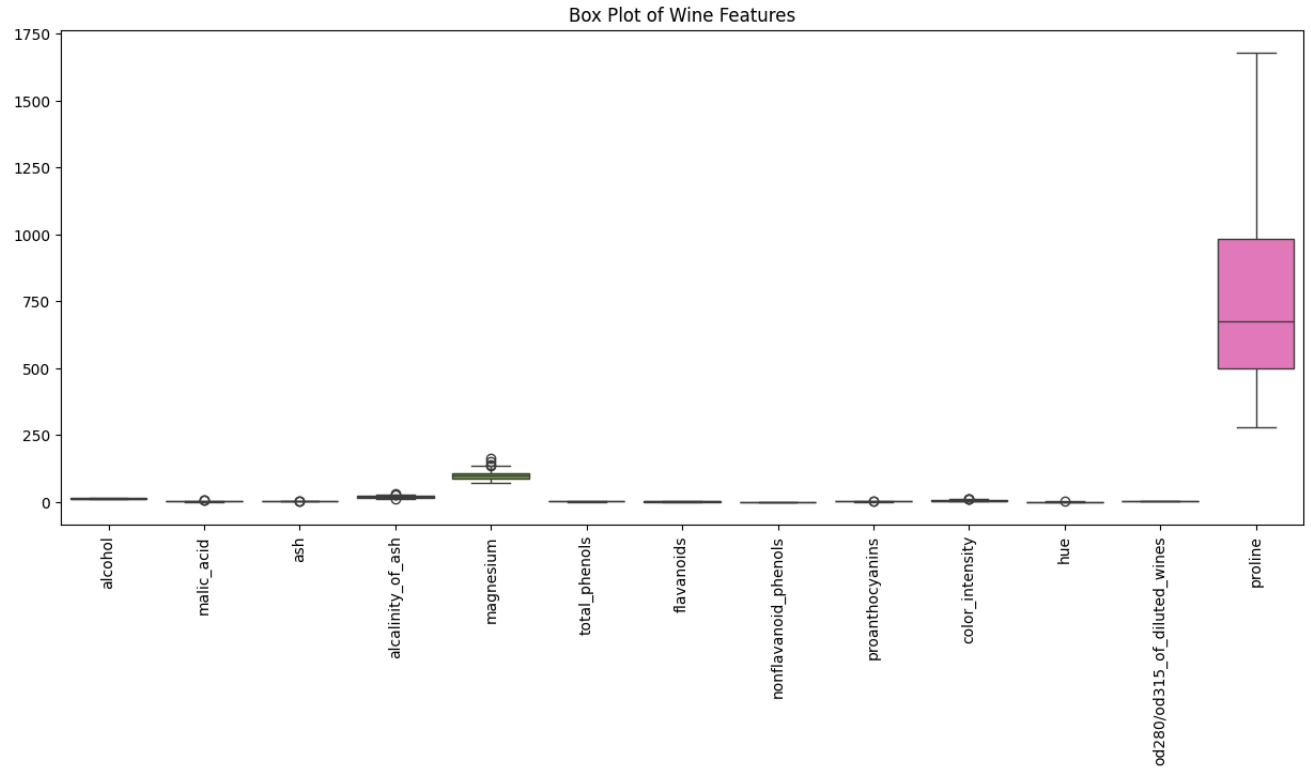

```
plt.figure(figsize=(15,6))

sns.boxplot(data=df.iloc[:, :-1])

plt.xticks(rotation=90)

plt.title("Box Plot of Wine Features")

plt.show()
```



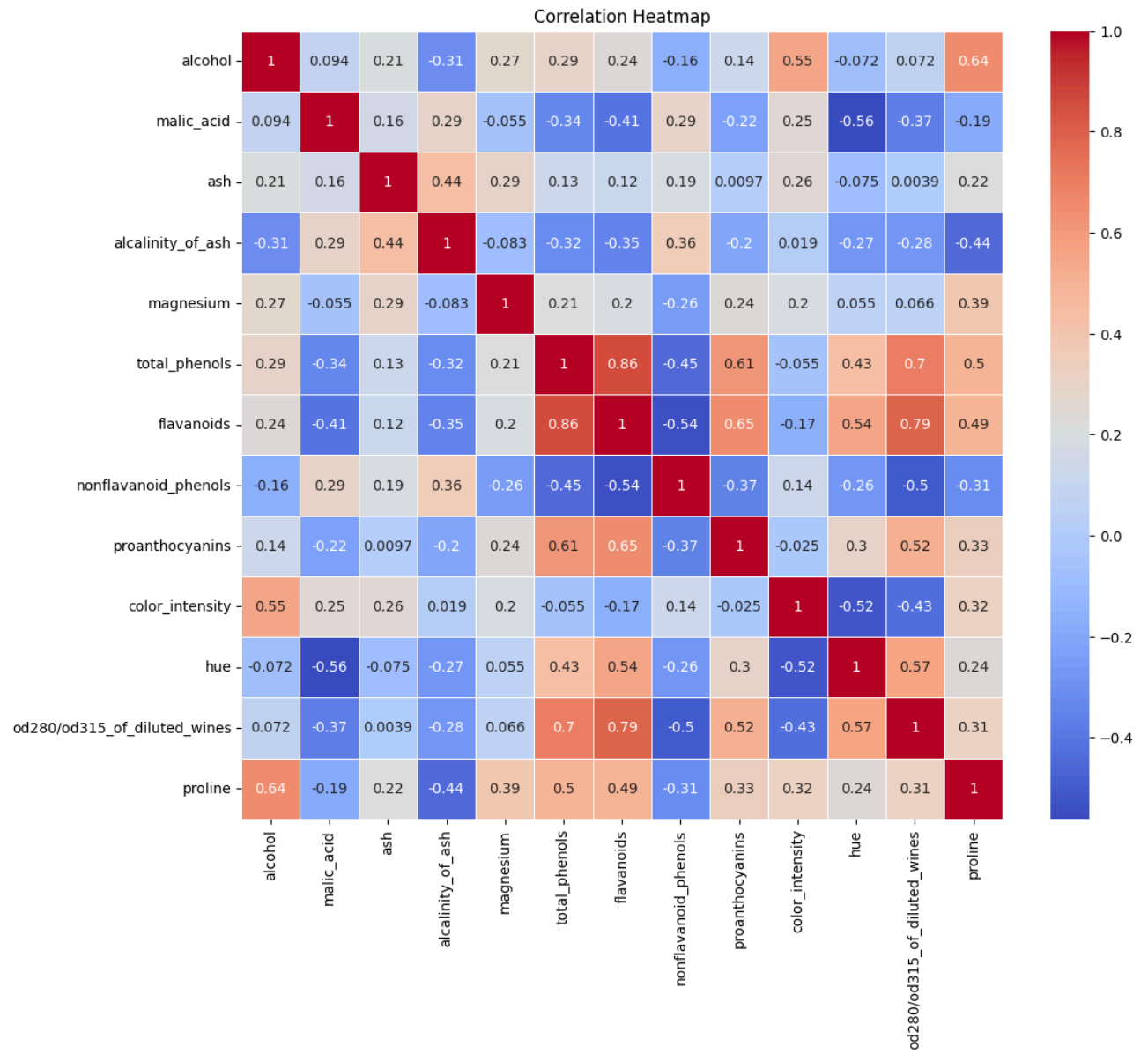

```
plt.figure(figsize=(12,10))

corr = df.iloc[:, :-1].corr()

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    linewidths=0.5
)

plt.title("Correlation Heatmap")

plt.show()
```




```
# Features
X = df.drop("Wine Class", axis=1)

# Target
y = df["Wine Class"]

print("Features Shape :", X.shape)
print("Target Shape   :", y.shape)
```

```
Features Shape : (178, 13)
Target Shape   : (178,)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
```

```
)

print("Training Samples :", X_train.shape[0])
print("Testing Samples  :", X_test.shape[0])
```

```
Training Samples : 142
Testing Samples  : 36
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

print("Feature Scaling Completed!")
```

```
Feature Scaling Completed!
```

```
#neural network
model = Sequential()

# Input Layer + Hidden Layer 1
model.add(
    Dense(
        units=32,
        activation="relu",
        input_shape=(13,)
    )
)

# Hidden Layer 2
model.add(
    Dense(
        units=16,
        activation="relu"
    )
)

# Output Layer
model.add(
    Dense(
        units=3,
        activation="softmax"
    )
)

print("Neural Network Created Successfully!")
```

```
Neural Network Created Successfully!
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	448
dense_1 (Dense)	(None, 16)	528
dense_2 (Dense)	(None, 3)	51

Total params: 1,027 (4.01 KB)
Trainable params: 1,027 (4.01 KB)

```
#model compile
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model Compiled Successfully!")
```

```
Model Compiled Successfully!
```

```
#train model
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.20,
    verbose=1
)
```

```
Epoch 1/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 3s 77ms/step - accuracy: 0.4336 - loss: 1.2015 - val_accuracy: 0.6897 - val_loss: 0.8937
Epoch 2/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.5133 - loss: 1.0657 - val_accuracy: 0.8276 - val_loss: 0.7814
Epoch 3/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.5664 - loss: 0.9570 - val_accuracy: 0.8276 - val_loss: 0.6874
Epoch 4/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.5841 - loss: 0.8670 - val_accuracy: 0.8621 - val_loss: 0.6149
Epoch 5/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.6903 - loss: 0.7938 - val_accuracy: 0.8621 - val_loss: 0.5469
Epoch 6/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.7699 - loss: 0.7207 - val_accuracy: 0.9310 - val_loss: 0.4802
Epoch 7/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.8761 - loss: 0.6526 - val_accuracy: 0.9310 - val_loss: 0.4240
Epoch 8/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.8938 - loss: 0.5861 - val_accuracy: 0.9310 - val_loss: 0.3730
Epoch 9/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9292 - loss: 0.5191 - val_accuracy: 0.9310 - val_loss: 0.3307
Epoch 10/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9646 - loss: 0.4493 - val_accuracy: 0.9310 - val_loss: 0.2919
Epoch 11/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9912 - loss: 0.3869 - val_accuracy: 0.9655 - val_loss: 0.2584
Epoch 12/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9912 - loss: 0.3308 - val_accuracy: 1.0000 - val_loss: 0.2289
Epoch 13/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9912 - loss: 0.2817 - val_accuracy: 1.0000 - val_loss: 0.2077
Epoch 14/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 1.0000 - loss: 0.2433 - val_accuracy: 1.0000 - val_loss: 0.1859
Epoch 15/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 1.0000 - loss: 0.2094 - val_accuracy: 1.0000 - val_loss: 0.1621
Epoch 16/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 1.0000 - loss: 0.1815 - val_accuracy: 1.0000 - val_loss: 0.1393
Epoch 17/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 1.0000 - loss: 0.1581 - val_accuracy: 1.0000 - val_loss: 0.1218
Epoch 18/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.1381 - val_accuracy: 1.0000 - val_loss: 0.1088
Epoch 19/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 1.0000 - loss: 0.1208 - val_accuracy: 1.0000 - val_loss: 0.0964
Epoch 20/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.1067 - val_accuracy: 1.0000 - val_loss: 0.0856
Epoch 21/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.0952 - val_accuracy: 1.0000 - val_loss: 0.0760
Epoch 22/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.0846 - val_accuracy: 1.0000 - val_loss: 0.0672
Epoch 23/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0757 - val_accuracy: 1.0000 - val_loss: 0.0597
Epoch 24/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0682 - val_accuracy: 1.0000 - val_loss: 0.0527
Epoch 25/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0618 - val_accuracy: 1.0000 - val_loss: 0.0466
Epoch 26/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0564 - val_accuracy: 1.0000 - val_loss: 0.0409
Epoch 27/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0510 - val_accuracy: 1.0000 - val_loss: 0.0370
Epoch 28/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0465 - val_accuracy: 1.0000 - val_loss: 0.0334
Epoch 29/50
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0426 - val_accuracy: 1.0000 - val_loss: 0.0306
```

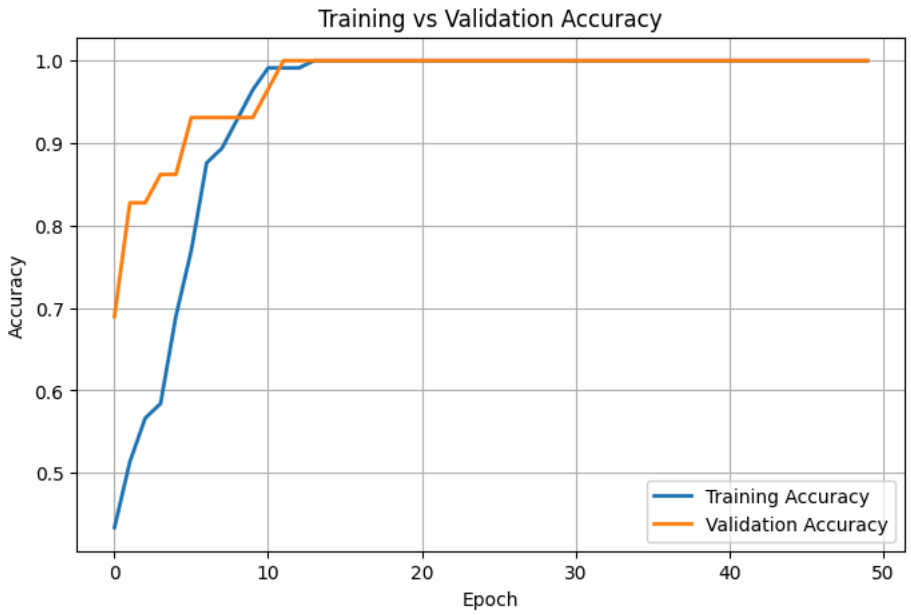
```
#training and validation accuracy graph
plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'], label='Training Accuracy', linewidth=2)
plt.plot(history.history['val_accuracy'], label='Validation Accuracy', linewidth=2)

plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
```

```
plt.grid(True)

plt.show()
```

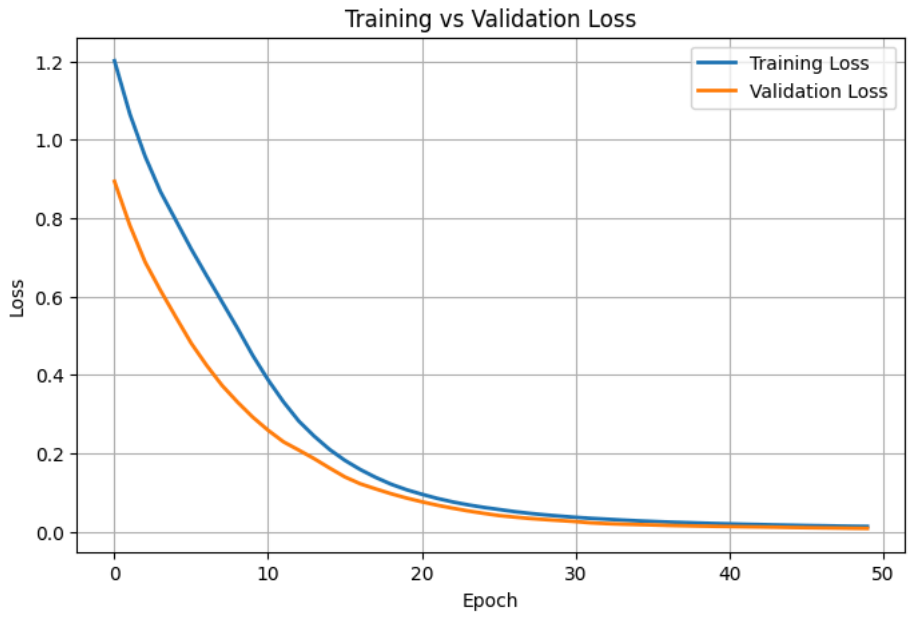



```
#training and validation loss graph
plt.figure(figsize=(8,5))

plt.plot(history.history['loss'], label='Training Loss', linewidth=2)
plt.plot(history.history['val_loss'], label='Validation Loss', linewidth=2)

plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```




```
#model evaluation

test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Loss :", round(test_loss,4))
print("Test Accuracy :", round(test_accuracy*100,2), "%")
```

```
Test Loss : 0.1008
Test Accuracy : 97.22 %
```

```
#predictions

y_pred_prob = model.predict(X_test)

y_pred = np.argmax(y_pred_prob, axis=1)

print("Predictions Completed Successfully!")
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 82ms/step
Predictions Completed Successfully!
```

```
#confusion matrix

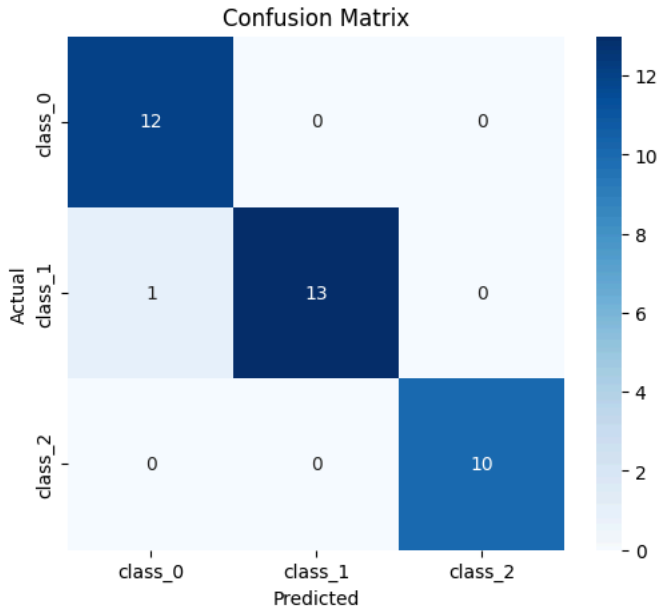
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=wine.target_names,
    yticklabels=wine.target_names
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```




```
#classification report

print(classification_report(
    y_test,
    y_pred,
    target_names=wine.target_names
))
```

```
              precision    recall  f1-score   support

     class_0       0.92      1.00      0.96        12
     class_1       1.00      0.93      0.96        14
     class_2       1.00      1.00      1.00        10

    accuracy                           0.97        36
```

```
   macro avg       0.97      0.98      0.97        36
weighted avg       0.97      0.97      0.97        36
```

```
#learning rate comparison

learning_rates = [0.1, 0.01, 0.001]

accuracies = []

for lr in learning_rates:

    temp_model = Sequential([
        Dense(32, activation='relu', input_shape=(13,)),
        Dense(16, activation='relu'),
        Dense(3, activation='softmax')
    ])

    temp_model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    temp_model.fit(
        X_train,
        y_train,
        epochs=50,
        batch_size=16,
        verbose=0
    )

    loss, acc = temp_model.evaluate(
        X_test,
        y_test,
        verbose=0
    )

    accuracies.append(acc*100)

comparison_df = pd.DataFrame({
    "Learning Rate": learning_rates,
    "Accuracy (%)": accuracies
})

comparison_df
```

	Learning Rate	Accuracy (%)
0	0.100	97.222221
1	0.010	97.222221
2	0.001	100.000000

```
#Learning rate vs accuracy graph

plt.figure(figsize=(10,6))

# Different colors for bars
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']

bars = plt.bar(
    [str(lr) for lr in learning_rates],
    accuracies,
    color=colors,
    edgecolor='black',
    linewidth=1.5,
    width=0.55
)

# Show accuracy value above each bar
for bar, acc in zip(bars, accuracies):
    plt.text(
        bar.get_x() + bar.get_width()/2,
        acc + 0.3,
        f"{acc:.2f}%",
```

```
        ha='center',
        fontsize=11,
        fontweight='bold'
    )

plt.title(
    "Effect of Learning Rate on Model Accuracy",
    fontsize=16,
    fontweight='bold'
)

plt.xlabel(
    "Learning Rate",
    fontsize=13,
    fontweight='bold'
)

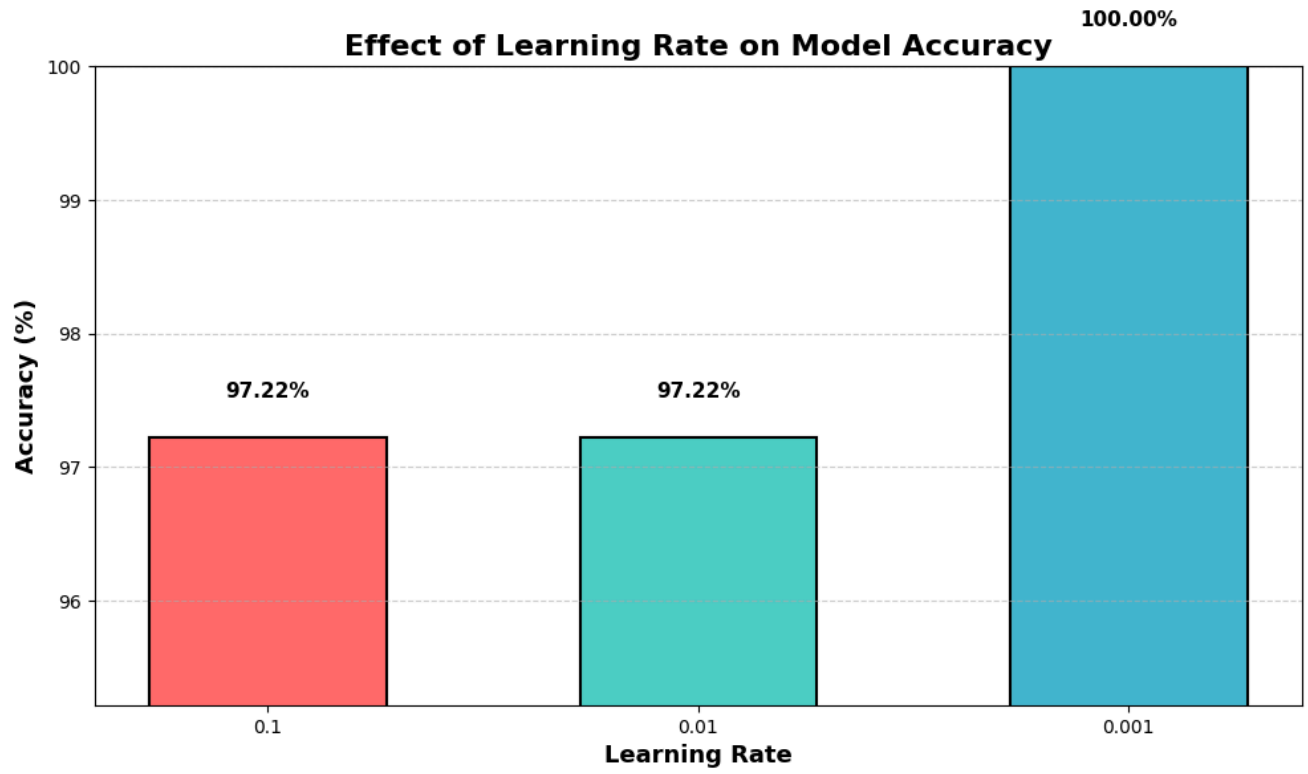
plt.ylabel(
    "Accuracy (%)",
    fontsize=13,
    fontweight='bold'
)

plt.ylim(min(accuracies)-2, 100)

plt.grid(axis='y', linestyle='--', alpha=0.6)

plt.tight_layout()

plt.show()
```




```
#epoch comparison

epochs_list = [20,50,100]

epoch_accuracy = []

for ep in epochs_list:

    temp_model = Sequential([
        Dense(32, activation='relu', input_shape=(13,)),
        Dense(16, activation='relu'),
        Dense(3, activation='softmax')
    ])
```

```
        temp_model.compile(
            optimizer=Adam(learning_rate=0.001),
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy']
        )

        temp_model.fit(
            X_train,
            y_train,
            epochs=ep,
            batch_size=16,
            verbose=0
```